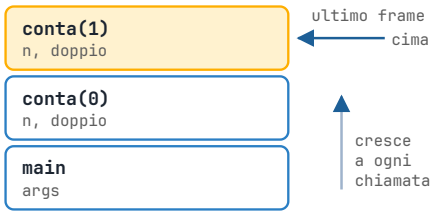
GESTIONE DELLA MEMORIA IN JAVA

Stack e heap · Approfondimento con esempi di codice · Java 25

Quando un programma Java è in esecuzione, la JVM organizza la memoria in zone con compiti diversi. Per capire che cosa succede a variabili e oggetti ne bastano due. Questo è il modello con cui ragionare sul codice. La JVM può ottimizzare l'esecuzione, ma il comportamento del programma resta quello descritto qui.

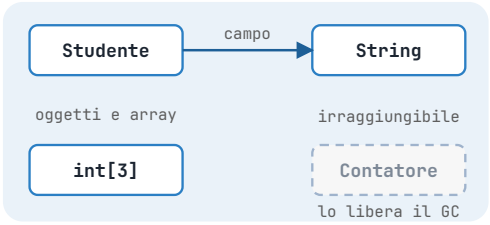
Stack

Lo stack è una pila: l'ultimo frame aggiunto è il primo a uscire. Aggiungere un frame è immediato, ma lo spazio è limitato: troppe chiamate causano `StackOverflowError`.



Heap

Area unica, condivisa da tutto il programma, per oggetti e array. Non si libera a mano: il garbage collector recupera ciò che non è più raggiungibile. Se lo spazio finisce: `OutOfMemoryError`.



01 Variabile locale primitiva

`Stack, nel frame del metodo`

Ogni chiamata di un metodo crea un **frame** nuovo, e il frame contiene le variabili locali di quella chiamata. Nella ricorsione lo si vede bene: lo stesso metodo è attivo tre volte, e ognuna ha la sua variabile `doppio`.

```
public class LocalePrimitiva {
    static void conta(int n) {
        int doppio = n * 2;  // una variabile per ogni chiamata
        if (n < 2) {
            conta(n + 1);  // nuovo frame sopra a questo
        }
        System.out.println("n=" + n + " doppio=" + doppio);
    }

    public static void main(String[] args) {
        conta(0);
    }
}
```

OUTPUT

```
n=2 doppio=4
n=1 doppio=2
n=0 doppio=0
```

Quando il metodo termina, il suo frame sparisce e con lui le variabili locali.

02 Parametro primitivo

`Stack, nel frame del metodo`

Un parametro è una variabile locale che riceve il valore al momento della chiamata. Il valore viene **copiato**: il metodo lavora sulla copia, e la variabile del chiamante non cambia.

```
public class ParametroPrimitivo {
    static void azzera(int x) {
        x = 0;  // cambia solo la copia
        System.out.println("dentro azzera: " + x);
    }

    public static void main(String[] args) {
        int voto = 28;
        azzera(voto);
        System.out.println("nel main: " + voto);
    }
}
```

OUTPUT

```
dentro azzera: 0
nel main: 28
```

Il parametro primitivo è una copia del valore.

03 Variabile dentro un blocco

`Stack, nello stesso frame del metodo`

Le graffe di un `if` o di un `for` non creano un frame: limitano solo lo **scope**, cioè la zona di codice in cui il nome esiste. Il frame resta quello del metodo.

```
public class Blocco {
    static int calcola(int a) {
        int risultato = a;
        if (a > 0) {
            int nelBlocco = a * 2;  // stesso frame di calcola
            risultato = nelBlocco + 1;
        }
        int dopo = risultato + 1;  // nelBlocco non esiste più
        return dopo;
    }

    public static void main(String[] args) {
        System.out.println(calcola(5));
    }
}
```

OUTPUT

```
12
```

Il blocco decide dove il nome è visibile, non dove sta la variabile.

04 Variabile locale di tipo classe

`Riferimento nello stack, oggetto nello heap`

La variabile non contiene l'oggetto: contiene un **riferimento** all'oggetto. Assegnare una variabile a un'altra copia il riferimento, quindi le due variabili indicano lo stesso oggetto. Sono due nomi per una cosa sola.

```
class Contatore {
    int valore;
}

public class LocaleClasse {
    public static void main(String[] args) {
        Contatore a = new Contatore();
        Contatore b = a;  // copia il riferimento, non l'oggetto
        b.valore = 5;
        System.out.println("a.valore = " + a.valore);

        b = new Contatore();  // ora b indica un altro oggetto
        b.valore = 9;
        System.out.println("a.valore = " + a.valore);
    }
}
```

OUTPUT

```
a.valore = 5
a.valore = 5
```

Più variabili possono indicare lo stesso oggetto.

05 Parametro di tipo classe

`Copia del riferimento nello stack`

Anche qui Java copia, ma quello che copia è il riferimento. Il metodo arriva allo **stesso oggetto** e può modificarlo. Se invece assegna al parametro un oggetto nuovo, cambia solo la sua copia del riferimento.

```
class Contatore {
    int valore;
}

public class ParametroClasse {
    static void modifica(Contatore c) {
        c.valore = 10;  // stesso oggetto del chiamante
    }

    static void sostituisci(Contatore c) {
        c = new Contatore();  // cambia solo la copia locale
        c.valore = 99;
    }

    public static void main(String[] args) {
        Contatore x = new Contatore();
        modifica(x);
        System.out.println(x.valore);
        sostituisci(x);
        System.out.println(x.valore);
    }
}
```

OUTPUT

```
10
10
```

L'oggetto è lo stesso, non viene copiato.

06 Oggetto creato con new

`Heap`

L'oggetto non appartiene al frame che lo ha creato, quindi **sopravvive** alla fine del metodo, purché qualcuno ne conservi il riferimento. Quando non lo indica più nessuno, il garbage collector può liberare la memoria.

```java
class Contatore {
    int valore;
}

public class OggettoNew {
    static Contatore crea() {
        Contatore locale = new Contatore();
        locale.valore = 7;
        return locale;  // il frame sparisce, l'oggetto no
    }

    public static void main(String[] args) {
        Contatore c = crea();
        System.out.println(c.valore);
        c = null;  // ora può essere raccolto dal GC
    }
}
```

OUTPUT

```
7
```

La variabile muore con il frame, l'oggetto quando nessuno lo indica più.

07 Variabile di istanza, anche primitiva

`Heap, dentro l'oggetto`

`valore` è un `int`, ma sta nello heap perché fa parte dell'oggetto. Conta **dove è dichiarata** la variabile, non il suo tipo. Ogni oggetto ha la propria copia, inizializzata al valore predefinito, e senza un oggetto la variabile non esiste.

```java
class Contatore {
    int valore;  // primitivo, ma nell'oggetto
}

public class VariabileIstanza {
    public static void main(String[] args) {
        Contatore a = new Contatore();
        Contatore b = new Contatore();
        System.out.println(a.valore);
        a.valore = 3;
        System.out.println(a.valore + " e " + b.valore);
    }
}
```

OUTPUT

```
0
3 e 0
```

Usare il campo senza un oggetto, da un metodo `static`, non compila:

```
error: non-static variable
valore cannot be
referenced from a static
context
```

Esiste solo dopo `new`, una copia per ogni oggetto.

08 Array, anche di int

Heap

In Java un array è un oggetto, anche quando contiene primitivi. Gli elementi stanno dentro l'array, nello heap, e valgono da subito il valore predefinito. La variabile contiene solo il riferimento: per questo un metodo che riceve un array può modificarne gli elementi.

```java
import java.util.Arrays;

public class ArrayPrimitivi {
    static void primoA99(int[] v) {
        v[0] = 99;  // stesso array del chiamante
    }

    public static void main(String[] args) {
        int[] numeri = new int[3];
        System.out.println(Arrays.toString(numeri));
        int[] alias = numeri;  // copia il riferimento
        alias[1] = 5;
        primoA99(numeri);
        System.out.println(Arrays.toString(numeri));
    }
}
```

OUTPUT

```
[0, 0, 0]
[99, 5, 0]
```

Gli elementi sono dentro l'array, nello heap.

09 Array di oggetti

Heap

L'array contiene **riferimenti**, non oggetti. Appena creato, ogni posizione vale `null`; gli oggetti vanno creati a parte, e ognuno ha il suo spazio nello heap. Due posizioni possono indicare lo stesso oggetto.

```java
class Studente {
    String nome;

    Studente(String nome) {
        this.nome = nome;
    }
}

public class ArrayOggetti {
    public static void main(String[] args) {
        Studente[] classe = new Studente[3];
        System.out.println(classe[0]);

        classe[0] = new Studente("Anna");
        classe[1] = classe[0];  // stesso oggetto, due posizioni
        classe[1].nome = "Marco";
        System.out.println(classe[0].nome);
    }
}
```

OUTPUT

```
null
Marco
```

L'array contiene riferimenti, ogni oggetto è a parte.

10 String

`Heap; i letterali nel pool`

Una `String` è un oggetto nello heap. I letterali uguali condividono un solo oggetto, nello **string pool**. `new String` crea invece un oggetto distinto. Le stringhe sono immutabili: «modificarne» una significa crearne un'altra.

```java
public class Stringhe {
    public static void main(String[] args) {
        String a = "ciao";
        String b = "ciao";  // stesso oggetto del pool
        String c = new String("ciao");
        System.out.println(a == b);
        System.out.println(a == c);
        System.out.println(a.equals(c));

        String d = a;
        a = a + "!";  // nuovo oggetto, d non cambia
        System.out.println(a + " " + d);
    }
}
```

OUTPUT

```
true
false
true
ciao! ciao
```

Per confrontare il contenuto si usa `equals`, non `==`.

11 Variabile static

`Con la classe, una sola copia`

Una variabile `static` appartiene alla classe, non ai singoli oggetti: ne esiste **una sola copia**, condivisa da tutti, ed è disponibile anche se non è stato creato nessun oggetto.

```java
class Biglietto {
    static int venduti = 0;  // una sola, per tutta la classe
    int numero;  // una per ogni oggetto

    Biglietto() {
        venduti = venduti + 1;
        numero = venduti;
    }
}

public class VariabileStatic {
    public static void main(String[] args) {
        System.out.println(Biglietto.venduti);
        Biglietto a = new Biglietto();
        Biglietto b = new Biglietto();
        System.out.println(a.numero + " " + b.numero);
        System.out.println(Biglietto.venduti);
    }
}
```

OUTPUT

```
0
1 2
2
```

Una sola copia per tutta l'esecuzione.

Riepilogo

Cosa	Dove	Da ricordare
Variabile locale primitiva	Stack, nel frame del metodo	Quando il metodo termina, il suo frame sparisce e con lui le variabili locali.
Parametro primitivo	Stack, nel frame del metodo	Il parametro primitivo è una copia del valore.
Variabile dentro un blocco	Stack, nello stesso frame del metodo	Il blocco decide dove il nome è visibile, non dove sta la variabile.
Variabile locale di tipo classe	Riferimento nello stack, oggetto nello heap	Più variabili possono indicare lo stesso oggetto.
Parametro di tipo classe	Copia del riferimento nello stack	L'oggetto è lo stesso, non viene copiato.
Oggetto creato con new	Heap	La variabile muore con il frame, l'oggetto quando nessuno lo indica più.
Variabile di istanza, anche primitiva	Heap, dentro l'oggetto	Esiste solo dopo `new`, una copia per ogni oggetto.
Array, anche di int	Heap	Gli elementi sono dentro l'array, nello heap.
Array di oggetti	Heap	L'array contiene riferimenti, ogni oggetto è a parte.
String	Heap; i letterali nel pool	Per confrontare il contenuto si usa `equals`, non `==`.
Variabile static	Con la classe, una sola copia	Una sola copia per tutta l'esecuzione.

Fonti

The Java Virtual Machine Specification (Java SE 25).